

With all these changes in place, it is time to see the results. Run the development server using `python manage.py runserver`, and then visit `http://localhost:8000/news/`. It should look something like figure 3.3.

You're much closer to having a real website now. Each web page you wish to host gets its own view. Each view gets a corresponding URL as a registered route and an HTML page. The HTML gets rendered from a template in your project's `RiffMates/RiffMates/templates` folder. However, there are two more things that will make your HTML writing easier: escaping special characters and composing templates. Let's start by dealing with those pesky special characters.



Figure 3.3 Your `news()` view

3.7 *Escaping special characters*

HTML uses angle brackets to make tags. Unfortunately, angle brackets can also function as greater-than and less-than signs. Your text may need to include these symbols, so HTML provides entities to work around this. If you want to write a greater than symbol in HTML, you use `>`.

Template variables are data. Data comes from all sorts of places and may contain characters special to HTML. To display your data on the page as it is meant to appear, a transformation has to happen. Django has your back and automatically does the transformation. This is known as *escaping characters*.

To play with escaping, you're going to need another template. Earlier, you used the Django REPL to directly instantiate a `Template` object; for historical reasons, this version of the object does not do auto-escaping. When you load a template file into the engine through the `render()` shortcut, you are actually using a different version of a `Template` object that does auto-escape. To build a realistic experiment, you need a little snippet of a template file in your `RiffMates/RiffMates/templates` directory—I named mine `experiment.html`. Create your own experimental file, like in the following listing. My version renders a single variable.

Listing 3.10 Mini-template `experiment.html` for playing with auto-escape

```
I love the sound a {{ instrument }} makes.
```

With this template in place, you can load and render it in the REPL. You're going to do that a bunch using different values of `instrument` to see the results of auto-escaping.